

Веб-сервер NginX

NginX

- Nginx (вимовляється як «двигун х») — це високопродуктивний веб-сервер з відкритим кодом та зворотний проксі-сервер.
 - Він розроблений для обробки високої паралельності та великомасштабних веб-сайтів і програм.
 - Nginx відомий своєю ефективною та подієво-орієнтованою архітектурою, яка дозволяє йому обробляти тисячі одночасних підключень з низьким використанням ресурсів.
-
- Nginx був створений російським інженером-програмістом Ігорем Сисоєвим у 2004 році.
 - Спочатку розроблений для вирішення проблеми C10K, яка стосується обробки 10 000 одночасних підключень.
 - Nginx швидко здобув популярність завдяки своїй швидкості, масштабованості та стабільності, і став одним з найбільш широко використовуваних веб-серверів у світі.

Важливість NginX

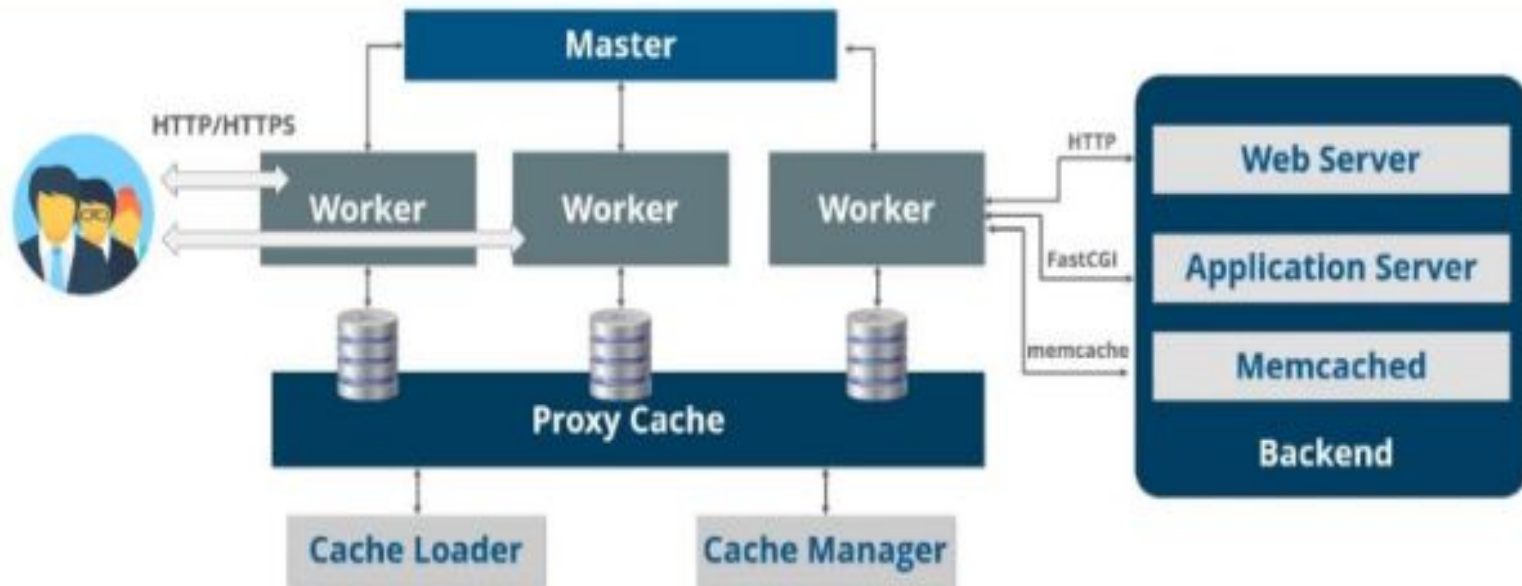
- **Висока продуктивність:** Nginx відомий своєю високопродуктивною архітектурою, що робить його придатним для обробки великого веб-трафіку та швидкої доставки контенту.
- **Масштабованість:** Подійно-керована модель Nginx дозволяє ефективно обробляти велику кількість одночасних підключень, що робить його ідеальним для веб-сайтів та програм з високим трафіком.
- **Зворотний проксі-сервер:** Nginx може виступати як зворотний проксі-сервер, розподіляючи вхідні запити на сервери сервера, покращуючи продуктивність та забезпечуючи балансування навантаження.
- **Завершення SSL/TLS:** Nginx підтримує шифрування SSL/TLS та може обробляти завершення SSL, розвантажуючи сервери сервера.
- **Кешування:** Nginx має вбудовані можливості кешування, що дозволяє йому обслуговувати статичний контент безпосередньо з кешу та зменшувати навантаження на сервери сервера.
- **Розширюваність:** Nginx має високу розширюваність і може бути розширений за допомогою модулів для додавання додаткових функцій та можливостей за потреби.

NginX проти Apache

Хоча Apache є популярним веб-сервером, Nginx пропонує певні переваги в певних сценаріях:

- Продуктивність: Nginx відомий своєю продуктивністю та ефективністю, особливо в обробці великої кількості одночасних підключень.
- Використання ресурсів: Nginx зазвичай використовує менше системних ресурсів порівняно з Apache, що робить його придатним для середовищ з обмеженими ресурсами.
- Зворотний проксі-сервер: Можливості зворотного проксі-сервера Nginx та функції балансування навантаження високо цінуються.
- Кешування: Вбудовані можливості кешування Nginx роблять його ефективним в обслуговуванні статичного контенту.

Архітектура NginX



Архітектура NginX

Архітектура, керована подіями, та асинхронна архітектура

- Nginx працює за подієвою моделлю, де він може ефективно керувати та обробляти події, не блокуючи та не витрачаючи системні ресурси.
- Асинхронна обробка дозволяє Nginx обробляти кілька одночасних з'єднань без потреби в окремому потоку чи процесі для кожного з'єднання.

Архітектура NginX

Робочі процеси

- Nginx використовує один або декілька робочих процесів для обробки вхідних підключень та запитів процесів.
- Кожен робочий процес може обробляти кілька підключень одночасно, максимізуючи використання системних ресурсів.
- Кількість робочих процесів можна налаштувати на основі доступних ядер процесора та очікуваного трафіку.

Архітектура NginX

Обробка з'єднань

- Nginx використовує ефективний механізм обробки з'єднань, керований подіями.
- Вхідні з'єднання приймаються та призначаються робочим процесам.
- Кожен робочий процес може одночасно обробляти кілька з'єднань за допомогою циклу подій.
- Цикл подій ефективно керує операціями вводу-виводу, такими як читання з сокетів та запис у них, без блокування та марнування ресурсів.
- Nginx використовує неблокуючі операції вводу-виводу та використовує механізми операційної системи, такі як `epoll` або `kqueue`, для сповіщення про події.
- Асинхронна обробка дозволяє Nginx ефективно обробляти велику кількість з'єднань з низьким навантаженням на процесор та пам'ять.

Конфігурація NginX

Файл nginx.conf

- Файл nginx.conf – це основний конфігураційний файл для Nginx.
- Він містить глобальні директиви конфігурації, включаючи налаштування для робочих процесів, ведення журналу помилок та інші параметри сервера.
- Він розділений на кілька розділів і може містити додаткові файли конфігурації.

Конфігурація NginX

Базові директиви конфігурації

Nginx надає різні директиви для налаштування своєї поведінки. Деякі поширені директиви включають:

- `worker_processes`: Визначає кількість робочих процесів.
- `error_log`: Встановлює місцезнаходження та рівень ведення журналу помилок.
- `events`: Налаштовує параметри, пов'язані з подіями, такі як модель подій та максимальна кількість підключень.

Конфігурація NginX

Блоки сервера

- Серверні блоки (також відомі як віртуальні хости) дозволяють розміщувати кілька веб-сайтів або програм на одному сервері Nginx.
- Кожен серверний блок представляє окрему конфігурацію веб-сайту або програми.
- Серверні блоки містять директиви, специфічні для цього конкретного веб-сайту або програми, такі як ім'я_сервера, кореневий каталог та розташування.

Конфігурація NginX

Щоб налаштувати блоки сервера в Nginx:

1. Створіть новий файл конфігурації або відредагуйте файл конфігурації за замовчуванням,
2. зазвичай розташований у `/etc/nginx/conf.d/`.
3. Визначте блок `server` за допомогою директиви `server`.
4. Вкажіть директиву `server_name` з доменом або іменем хоста для
5. віртуального хоста.
6. Встановіть директиву `root`, щоб визначити кореневий каталог для віртуального хоста.
7. Додайте додаткові директиви та розташування, що відповідають вимогам віртуального хоста.

Конфігурація NginX

Виправлення неполадок

Під час роботи з віртуальними хостами Nginx ви можете зіткнутися з деякими поширеними проблемами:

- Неправильно налаштовано ім'я_сервера: Переконайтеся, що директива `server_name`
- правильно встановлена відповідно до домену або імені хоста.
- Дозволи на доступ до файлів і каталогів: Перевірте, чи кореневий каталог і його
- вміст мають відповідні дозволи для доступу Nginx.
- Конфлікти блоків сервера: Переконайтеся, що блоки сервера не перекриваються та не
- конфліктують один з одним.
- Перезавантаження служби Nginx: Після внесення змін перезавантажте службу Nginx,
- щоб застосувати нову конфігурацію.

Конфігурація NginX

Виправлення неполадок

- Журналування та повідомлення про помилки: Перевірте журнал помилок Nginx (розташування за замовчуванням: `/var/log/nginx/error.log`) на наявність відповідних повідомлень про помилки або попереджень.
- Синтаксичні помилки конфігурації: Перевірте конфігурацію Nginx на наявність синтаксичних помилок за допомогою команди «`sudo nginx -t`» перед повторним завантаженням конфігурації.
- Перегляньте документацію та онлайн-ресурси: Зверніться до документації Nginx та онлайн-форумів для отримання конкретних кроків та рішень усунення несправностей.



NginX + Apache

NginX + Apache

Nginx та Apache – це два популярні веб-сервери, які можна використовувати разом у конфігурації, відомій як Nginx + Apache або конфігурація "зворотного проксі".

Ця конфігурація поєднує сильні сторони обох серверів для досягнення кращої продуктивності, масштабованості та гнучкості.

Переваги NginX + Apache

1. Покращена продуктивність:

- Nginx може ефективно обслуговувати статичний контент, розвантажуючи Apache обробкою статичних файлів та звільняючи його ресурси для обробки динамічних запитів.
- Apache може зосередитися на виконанні динамічних скриптів та обробці складної логіки.

2. Покращена масштабованість:

- Nginx діє як зворотний проксі-сервер, розподіляючи вхідні запити між кількома серверами Apache.
- Це дозволяє горизонтальне масштабування шляхом додавання більшої кількості серверів Apache для обробки збільшеного трафіку.

3. Гнучкість у виконанні різних робочих навантажень:

- Nginx можна налаштувати для обробки певних типів запитів, таких як обслуговування статичних файлів або робота в якості балансувальника навантаження, тоді як Apache зосереджується на логіці програми.
- Такий розподіл обов'язків дозволяє кожному серверу виконувати свої завдання ефективніше.

Типові випадки використання для налаштування Nginx + Apache

1. Балансування навантаження:

Nginx може розподіляти вхідні запити між кількома серверами Apache, покращуючи продуктивність та доступність.

2. Кешування та прискорення:

Nginx може кешувати статичні файли або відповіді від бекенд-серверів Apache, зменшуючи навантаження на Apache та покращуючи час відгуку.

3. Припинення дії SSL-сертифіката:

Nginx може обробляти SSL-термінацію, розшифровувати вхідні HTTPS-запити та пересилати їх до Apache через звичайний HTTP, зменшуючи обчислювальне навантаження на Apache.

4. Зворотний проксі-сервер для бекенд-сервісів:

Nginx може виступати зворотним проксі-сервером для різних бекенд-сервісів, таких як Node.js, Ruby або Python-додатки, спрямовуючи запити на відповідний бекенд-сервер.

Проксі-сервери

Переадресація проксі-сервера

- Прямий проксі-сервер, також відомий як клієнтський проксі-сервер, розташований між клієнтами та Інтернетом.
- Клієнти надсилають запити на сервер прямого проксі-сервера, який потім пересилає запити до Інтернету від імені клієнтів.
- Типові випадки використання прямого проксі-сервера включають доступ до обмеженого контенту, покращення конфіденційності та кешування контенту.

Зворотний проксі

- Зворотний проксі-сервер, також відомий як серверний проксі-сервер, розташований між клієнтами та серверами.
- Клієнти надсилають запити до зворотного проксі-сервера, який потім пересилає запити на відповідні внутрішні сервери.
- Зворотні проксі-сервери можуть виконувати різні завдання, такі як балансування навантаження, завершення SSL-підключення,

Проксі-сервери. Зворотний проксі-сервер

Переваги використання зворотного проксі-сервера:

- а. Балансування навантаження: Розподіляє вхідні запити між кількома серверами backend для покращення продуктивності та обробки високого трафіку.
- б. Термінація SSL: Обробляє шифрування та дешифрування SSL, розвантажуючи обчислювальне навантаження з серверів backend.
- в. Кешування: Кешує статичний контент або відповіді від серверів backend, зменшуючи навантаження на сервери backend та покращуючи час відгуку.

Балансування навантаження

Балансування навантаження – це метод, який використовується для розподілу вхідного мережевого трафіку між кількома серверами з метою оптимізації використання ресурсів та підвищення продуктивності.

Балансувальники навантаження можуть бути реалізовані за допомогою апаратних або програмних рішень.

До поширених алгоритмів балансування навантаження належать Round Robin, Least Connection та IP Hash.

Переваги балансування навантаження:

1. Покращена масштабованість: Розподіляє запити між кількома серверами, що дозволяє горизонтальне масштабування для обробки збільшеного трафіку.
2. Висока доступність: Якщо один сервер виходить з ладу, балансувальники навантаження можуть автоматично перенаправляти запити на доступні сервери, забезпечуючи безперебійне обслуговування.

Налаштування Nginx як зворотного проксі-сервера

- Nginx може виступати в ролі зворотного проксі-сервера для Apache, обробляючи запити клієнтів та пересилаючи їх на сервери Apache.
- Така конфігурація може допомогти покращити продуктивність, масштабованість та забезпечити додаткові функції, такі як балансування навантаження та кешування.

Налаштування Nginx як зворотного проксі-сервера

Щоб налаштувати Nginx як зворотний проксі-сервер для Apache:

1. Відкрийте файл конфігурації Nginx (nginx.conf), розташований у /etc/nginx/.
2. Визначте новий блок сервера для конфігурації зворотного проксі-сервера.
3. Встановіть директиву server_name на домен або ім'я хоста, для якого Nginx діятиме як зворотний проксі-сервер.
4. Використовуйте директиву proxy_pass, щоб вказати URL-адресу або IP-адресу бекенд-сервера Apache.

Налаштування Nginx як зворотного проксі-сервера

Передача HTTP-заголовків

За замовчуванням Nginx передає деякі HTTP-заголовки від клієнта до бекенд-сервера Apache.

Однак деякі заголовки, такі як X-Forwarded-For та X-Real-IP, можуть потребувати явного налаштування в Nginx.

Налаштування Nginx як зворотного проксі-сервера

Виправлення неполадок

1. Неправильна URL-адреса `proxy_pass`: Перевірте, чи є URL-адреса, зазначена в директиві `proxy_pass`, правильною та доступною.
2. Невідповідні порти: Переконайтеся, що зворотний проксі-сервер Nginx та сервер Apache прослуховують один і той самий порт.
3. Перенаправлення HTTP/HTTPS: Налаштуйте конфігурацію Nginx для обробки перенаправлення HTTP на HTTPS або навпаки.
4. Таймаут проксі-сервера: Збільште таймаут проксі-сервера, якщо запити Apache до сервера обробляються довше.
5. Журнал та повідомлення про помилки: Перевірте журнал помилок Nginx (розташування за замовчуванням: `/var/log/nginx/error.log`) на наявність будь-яких відповідних повідомлень про помилки або попереджень.
6. Синтаксичні помилки конфігурації: Перевірте конфігурацію Nginx на наявність синтаксичних помилок за допомогою команди `'sudo nginx -t'` перед перезавантаженням конфігурації.
7. Перегляньте документацію та онлайн-ресурси: Зверніться до документації та онлайн-форумів Nginx та Apache для отримання конкретних кроків та рішень усунення несправностей.

Віртуальні хости Nginx та Apache

Як Nginx, так і Apache підтримують віртуальні хости, що дозволяє розміщувати кілька веб-сайтів на одному сервері.

Однак, існують деякі відмінності в налаштуванні та управлінні віртуальними хостами між цими двома серверами.

Apache	NginX
Використовує концепцію контейнерів VirtualHost у <code>httpd.conf</code> або окремих файлах конфігурації.	Використовує блоки сервера у файлі <code>nginx.conf</code> або окремих файлах конфігурації в каталозі <code>conf.d</code> .
Кожен блок VirtualHost містить директиви конфігурації, специфічні для певного домену або вебсайту.	Кожен блок сервера визначає конфігурацію для певного домену або вебсайту.

Поради щодо керування складними налаштуваннями

1. Правила іменування:

Використовуйте описові назви для віртуальних хостів, що полегшить їх ідентифікацію та керування ними у файлах конфігурації.

2. Окремі файли конфігурації:

- Розділіть конфігурації віртуального хоста на окремі файли для кращої організації та легшого обслуговування.
- Включіть ці файли до основної конфігурації за допомогою директиви 'include'.

3. Шаблони сервера:

- Створюйте шаблони серверів багаторазового використання зі спільними параметрами конфігурації, щоб спростити налаштування нових віртуальних хостів.
- Використовуйте змінні та включення, щоб уникнути дублювання схожих блоків конфігурації.

Поради щодо поширених помилок та як їх уникнути

1. Конфлікт імен серверів:

Переконайтеся, що імена серверів або доменні імена, що використовуються в конфігураціях віртуального хоста, не конфліктують одне з одним.

2. Конфлікти портів:

Перевірте, чи віртуальні хости призначені різним портам або IP-адресам, щоб уникнути конфліктів.

3. Порядок блокування сервера:

Nginx обробляє блоки сервера зверху вниз. Переконайтеся, що потрібний блок сервера вказаний першим для обробки запиту.

4. Синтаксичні помилки конфігурації:

Регулярно перевіряйте файли конфігурації на наявність синтаксичних помилок за допомогою команди `'sudo nginx -t'` або `'sudo apachectl -t'` перед повторним завантаженням конфігурації.



Переадресація посилань (переадресації)

Перенаправлення URL-адрес

- Перенаправлення URL-адрес – це процес переадресації однієї URL-адреси на іншу.
- Це допомагає перенаправляти користувачів або пошукові системи з однієї веб-сторінки на іншу.
- Корисно для керування змінами на веб-сайті, оптимізації SEO та покращення взаємодії з користувачем.

Типи перенаправлень

301 Перенаправлення (постійне)

- А. Надсилає браузеру код стану «301 Переміщено назавжди».
- В. Доручає пошуковим системам індексувати нову URL-адресу.
- С. Рекомендовано для постійних змін URL-адрес.

302 Переадресація (тимчасова)

- А. Надсилає браузеру код стану «302 Знайдено».
- В. Наказує пошуковим системам зберігати оригінальну URL-адресу індексованою.
- С. Корисно для тимчасових переадресацій або А/В-тестування.

307 Перенаправлення (тимчасове)

- А. Надсилає браузеру код стану "307 Тимчасове перенаправлення".
- В. Наказує пошуковим системам зберігати оригінальну URL-адресу індексованою.
- С. Подібно до перенаправлення 302, але явно вказує на тимчасове перенаправлення.

308 Перенаправлення (постійне)

- А. Надсилає браузеру код стану "308 Permanent Redirect".
- В. Дає інструкцію пошуковим системам індексувати нову URL-адресу.
- С. Введено як заміну перенаправленню 301, щоб уникнути потенційних проблем із кешуванням.

Налаштування переадресацій в Apache

1. Використання файлу .htaccess:

- Знайдіть або створіть файл .htaccess у кореневому каталозі вебсайту.
- Додайте правило перенаправлення за допомогою директиви Redirect.

Приклад: Перенаправлення 301 /стара-сторінка.html /нова-сторінка.html

1. Використання віртуального хоста Apache:

- Відредагуйте файл конфігурації віртуального хоста.
- Додайте директиву RedirectMatch у блок віртуального хоста.

Приклад: RedirectMatch 301 ^/category/(.*)\$ [https://example.com/newcategory/\\$1](https://example.com/newcategory/$1)

Налаштування переадресацій у NginX

1. Використання блоку сервера:

- Відредагуйте файл конфігурації блоку сервера Nginx.
- Додайте правило перенаправлення за допомогою директиви return.
Приклад: повернути 301 /стара-сторінка.html <https://example.com/нова-сторінка.html>;

1. Використання блоку розташування:

- Відредагуйте файл конфігурації Nginx.
- Додайте блок розташування та використовуйте директиву rewrite для перенаправлення.
Приклад:

```
location /category/ {  
    rewrite ^/category/(.*)$ https://example.com/new-category/\$1 permanent;  
}
```



Списки контролю доступу (ACL)

Налаштування перенаправлень у NginX

- Списки контролю доступу (ACL) забезпечують більш детальний рівень контролю доступу, ніж традиційні дозволи.
- ACL розширюють стандартні дозволи Unix, дозволяючи користувачам встановлювати певні дозволи для окремих користувачів або груп на файл чи каталог.
- ACL визначають додаткові права доступу, такі як читання, запис, виконання, видалення тощо.

Як працює ACL на веб-серверах

- Веб-сервери використовують ACL для керування доступом до файлів і каталогів на сервері.
- ACL дозволяють детально контролювати дозволи доступу для різних користувачів або груп.
- Їх можна використовувати для надання або заборони певних дозволів певним користувачам або групам.

Приклади конфігурацій ACL

1. Налаштування ACL для каталогу:

- Команда: `setfacl -m u:john:rwX /var/www/html/directory`
- Пояснення: Надає користувачеві "john" дозволи на читання, запис та виконання в каталозі.

2. Налаштування ACL для файлу:

- Команда: `setfacl -m g:marketing:rx /var/www/html/file.html`
- Пояснення: Надає групі "marketing" дозволи на читання та виконання файлу.

3. Зміна запису ACL:

- Команда: `setfacl -m u:sarah:rw /var/www/html/directory`
- Пояснення: Змінює запис ACL, щоб надати користувачеві "sarah" права на читання та запис.

Приклади конфігурацій ACL

4. Видалення запису ACL:

- Команда: `setfacl -x u:sarah /var/www/html/directory`
- Пояснення: Видаляє запис ACL для користувача "sarah" з каталогу.

5. Копіювання ACL з одного файлу в інший:

- Команда: `getfacl /var/www/html/file.html | setfacl -R --set-file=- /var/www/html/newfile.html`
- Пояснення: Рекурсивно копіює ACL з "file.html" до "newfile.html".

6. Перегляд ACL:

- Команда: `getfacl /var/www/html/directory`
- Пояснення: Відображає записи ACL для каталогу.



Базова автентифікація (AUTH)



Що таке базова автентифікація

- Базова автентифікація – це простий механізм автентифікації, який використовується для захисту веб-ресурсів.
- Він вимагає від користувачів введення імені користувача та пароля для доступу до захищеного контенту.
- Облікові дані надсилаються у звичайному тексті, тому рекомендується використовувати базову автентифікацію через HTTPS.

Як працює базова автентифікація

- Коли користувач намагається отримати доступ до захищеного ресурсу, сервер відповідає кодом стану «401 Неавторизовано».
- Сервер включає заголовок «WWW-Authenticate» з назвою області, щоб ідентифікувати захищену область.
- Браузер користувача запитує облікові дані та надсилає їх на сервер у заголовку «Авторизація».
- Якщо облікові дані збігаються, сервер відповідає кодом стану «200 ОК» та дозволяє доступ до ресурсу.

Реалізація базової AUTH в Apache

1. Увімкніть модуль автентифікації:

Приклад команди: `a2enmod auth_basic`

2. Створіть файл паролів:

Приклад команди: `htpasswd -c /etc/apache2/.htpasswd username`

3. Налаштуйте Apache для використання базової автентифікації:

Приклад конфігурації:

```
<Каталог /var/www/html/protected>
```

```
<Directory /var/www/html/protected>
```

```
AuthType Basic
```

```
AuthName "Restricted Content"
```

```
AuthUserFile /etc/apache2/.htpasswd
```

```
Require valid-user
```

```
</Directory>
```

Реалізація базової AUTH в Nginx

1. Створіть файл паролів:

Приклад команди: `htpasswd -c /etc/nginx/.htpasswd ім'я користувача`

2. Налаштуйте Nginx для використання базової автентифікації:

Приклад конфігурації:

```
location /protected {
```

```
    auth_basic "Restricted Content";
```

```
    auth_basic_user_file /etc/nginx/.htpasswd;
```

```
}
```



Кешування статичного кешу

Основи кешування статичного контенту

- Кешування статичного контенту – це процес зберігання статичних файлів (наприклад, зображень, CSS, JavaScript) у кеші для покращення продуктивності веб-сайту.
- Кешування зменшує навантаження на сервер і час, необхідний для доставки контенту користувачам.
- Статичні файли зберігаються в кеші та надаються безпосередньо користувачеві без необхідності повторного отримання із сервера.

Налаштування статичного кешу в Apache

1. Увімкніть модуль `mod_cache`:

Приклад команди: `a2enmod cache`

2. . Налаштуйте директиви кешування:

- Встановіть каталог `CacheRoot` для визначення місця розташування кешу.
- Використовуйте директиву `CacheEnable`, щоб вказати, які файли або каталоги кешувати.
- Встановіть директиву `CacheDefaultExpire`, щоб визначити час закінчення терміну дії кешованих файлів.

Налаштування статичного кешу в NginX

3. Увімкніть кешування в Nginx:

- Додайте директиву "proxy_cache_path", щоб вказати розташування та налаштування кешу.

4. Налаштуйте директиви кешування:

- Використовуйте блок "location", щоб вказати файли або каталоги для кешування.
- Додайте директиву "proxy_cache", щоб увімкнути кешування для вказаного розташування.
- Встановіть директиву "proxy_cache_valid", щоб визначити час дії кешованих файлів.

Переваги кешування статичного контенту

- Покращена продуктивність веб-сайту: кешування зменшує навантаження на сервер і швидше доставляє контент користувачам.
- Зменшення використання пропускну здатності: кешовані файли обслуговуються з кешу, а не завантажуються із сервера, що економить пропускну здатність.
- Покращений користувацький досвід: швидше завантаження забезпечує кращий досвід перегляду для користувачів.

Потенційні проблеми зі статичним кешуванням контенту

- Анулювання кешу: під час оновлення статичних файлів кеш потрібно анулювати, щоб користувачі отримували найновіший вміст.
- Узгодженість кешу: кешування може призвести до невідповідної роботи, якщо різні користувачі бачать різні версії кешованих файлів.
- Використання дискового простору: кешування статичних файлів вимагає дискового простору, тому важливо контролювати та керувати розміром кешу, щоб запобігти надмірному використанню диска.



Основи ведення журналу

Важливість ведення журналу для веб-серверів

- Ведення журналу є важливим аспектом адміністрування веб-сервера з різних причин.
- Журнали надають цінну інформацію про активність сервера, помилки та події безпеки.
- Вони допомагають у вирішенні проблем, моніторингу продуктивності сервера та проведенні аудитів.

Типи журналів: Журнали доступу

- Журнали доступу записують інформацію про вхідні запити до веб-сервера.
- Вони фіксують такі деталі, як IP-адреса запиту, запитувана URL-адреса, агент користувача та коди стану відповіді.
- Журнали доступу корисні для аналізу трафіку веб-сайту, виявлення тенденцій та усунення несправностей, пов'язаних із певними запитами.

Типи журналів: Журнали помилок

- Журнали помилок записують інформацію про помилки, що виникли на веб-сервері.
- Вони фіксують такі деталі, як тип помилки, позначка часу та відповідні повідомлення про помилки.
- Журнали помилок мають вирішальне значення для виявлення та вирішення проблем на стороні сервера, таких як помилки конфігурації, помилки програм та збої сервера.

Налаштування логування в Apache

1. Увімкнути модулі логування:

Приклади команд: `a2enmod log_config` (для базового логування), `a2enmod logio` (для розширеного логування)

2. Налаштуйте журнали доступу:

Приклад директиви: `CustomLog /var/log/apache2/access.log combined`

3. Налаштуйте журнали помилок:

Приклад директиви: `ErrorLog /var/log/apache2/error.log`

Налаштування логування в NginX

1. Увімкніть налаштування журналів доступу:

Приклад директиви: `access_log /var/log/nginx/access.log combined`

2. Налаштуйте журнали помилок:

Приклад директиви: `error_log /var/log/nginx/error.log error`

Розташування файлів журналів

- Журнали увімкнення доступу та журнали помилок в Apache зазвичай розташовані в каталозі `/var/log/apache2/`.
- Журнали доступу та журнали помилок у Nginx зазвичай розташовані в `/var/log/nginx/directory`.

Ротація та керування файлами журналів

- Файли журналів можуть з часом збільшуватися в розмірі, тому важливо впроваджувати ротацію журналів.
- Ротація журналів передбачає стиснення та архівування старих файлів журналів для економії місця на диску.
- Такі інструменти, як `logrotate`, можна використовувати для автоматизації завдань ротації та керування журналами.



Журнали доступу

Розуміння структури журналів доступу

Журнали доступу фіксують інформацію про вхідні запити до веб-сервера.

Кожен запис у журналі доступу представляє один запит.

Структура запису журналу доступу зазвичай включає такі поля:

- IP-адреса: IP-адреса клієнта, який робить запит.
- Мітка часу: Дата та час отримання запиту.
- Метод HTTP: Метод HTTP, що використовується в запиті (наприклад, GET, POST).
- Запитана URL-адреса: Шлях URL-адреси та параметри, запитувані клієнтом.
- Код стану HTTP: Код стану, повернутий сервером (наприклад, 200, 404).
- Агент користувача: Браузер або програма клієнта, яка робить запит.

Аналіз журналів доступу

Журнали доступу надають цінну інформацію про трафік веб-сайту та поведінку користувачів.

Аналіз журналів доступу може допомогти визначити популярні сторінки, виявити аномалії та усунути проблеми.

До поширених методів аналізу належать:

- Визначення основних джерел переходів та агентів користувача.
- Аналіз кодів статусу відповідей на наявність помилок.
- Вивчення шаблонів запитів, таких як часті або підозрілі запити.
- Моніторинг тенденцій трафіку з плином часу.

Налаштування журналів доступу в Apache

1. Увімкнути модуль логування:

Приклад команди: `a2enmod log_config`

1. Налаштуйте журнали доступу в Apache:

Приклад директиви: `CustomLog /var/log/apache2/access.log combined`

1. Поширені формати журналів в Apache:

- комбінований: Включає найчастіше використовувані поля журналу.
- загальний: Включає підмножину полів журналу

Налаштування журналів доступу в NginX

1. Увімкніть налаштування журналів доступу в Nginx:

Приклад директиви: `access_log /var/log/nginx/access.log combined`

1. Поширені формати журналів у Nginx:

- комбінований: Подібний до комбінованого формату журналів Apache.
- основний: Містить підмножину полів журналу.



Журнали помилок

Розуміння структури журналів помилок

Журнали помилок містять інформацію про помилки на стороні сервера та проблеми, з якими зіткнувся веб-сервер.

Кожен запис у журналі помилок представляє певну помилку або попередження.

Структура запису в журналі помилок зазвичай включає такі поля:

- Мітка часу: Дата та час виникнення помилки.
- Рівень помилки: Рівень серйозності помилки (наприклад, помилка, попередження, повідомлення).
- Повідомлення про помилку: Описове повідомлення з детальною інформацією про помилку.
- Відповідний контекст: Додаткова інформація, пов'язана з помилкою, така як назви файлів або номери рядків.

Аналіз журналів помилок

Журнали помилок є важливими для діагностики та усунення несправностей на стороні сервера.

Аналіз журналів помилок може допомогти виявити проблеми, знайти джерело помилок та вирішити проблеми.

Поширені методи аналізу включають:

- Виявлення повторюваних помилок або попереджень.
- Вивчення повідомлень про помилки та пов'язаного з ними контексту.
- Співвіднесення помилок з певними подіями або діями користувача.
- Моніторинг шаблонів помилок для виявлення тенденцій або проблем, що виникають.

Налаштування журналів помилок в Apache

1. Увімкнути модуль логування:

Приклад команди: `a2enmod log_config`

1. Налаштуйте журнали помилок в Apache:

Приклад директиви: `ErrorLog /var/log/apache2/error.log`

1. Додаткові параметри конфігурації:

- Директива `LogLevel`: Визначає мінімальний рівень серйозності для реєстрування.
- Директива `ErrorDocument`: Налаштовує сторінки помилок, що відображаються користувачам.

Налаштування журналів помилок у NginX

1. Налаштуйте журнали помилок у Nginx:

Приклад директиви: `error_log /var/log/nginx/error.log error`

1. Додаткові параметри конфігурації:

- Параметри директиви `error_log`: Визначає формат журналу та рівень серйозності.
- Директива `log_not_found`: Реєструє помилки для відсутніх файлів.



Інтерпретація журналів

Поширені записи журналу та їх значення

Записи журналу надають цінну інформацію про системні події, помилки та дії. Розуміння поширених записів журналу допомагає у вирішенні проблем та їх виявленні. Ось кілька прикладів поширених записів журналу та їх значень:

[INFO] – Інформаційне повідомлення про звичайну системну подію або активність.

[WARNING] – Вказує на потенційну проблему або аномальний стан, який слід контролювати.

[ERROR] – Вказує на помилку або збій, що потребує уваги або розслідування.

[CRITICAL] – Вказує на критичну помилку або збій, що потребує негайного усунення.

Інструменти для аналізу журналів

Для аналізу та інтерпретації файлів журналів у Linux доступні різні інструменти.

Ці інструменти надають функції для пошуку, фільтрації та візуалізації даних журналів для легшого аналізу.

Деякі поширені інструменти для аналізу журналів включають:

- `grep`: Інструмент командного рядка для пошуку файлів журналів на основі шаблонів.
- `awk`: Потужна мова сценаріїв для обробки даних журналів.
- `sed`: Редактор потоків для зміни файлів журналів на основі шаблонів.
- `tail`: Команда для відображення останніх кількох рядків файлу журналу.
- `logwatch`: Інструмент аналізу та звітності журналів, який підсумовує дані журналів.
- `ELK Stack` (Elasticsearch, Logstash, Kibana): Комплексне рішення для аналізу журналів.

Питання та відповіді